A PROJECT REPORT ON

A FULL-STACK CRUD APPLICATION

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY,
PUNE,IN THE PARIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE AWARD OF THE DEGREE

OF

THIRD YEAR COMPUTER ENGINEERING

BY

MAYUR UMAKANT ZOPE

UNDER THE GUIDANCE OF

PROF. SONU KHAPEKAR

Nutan Maharashtra Vidya Prasarak Mandal's
NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY, PUNE

Samarth Vidya Sankul, Vishnupuri,
Talegaon Dabhade, Maharashtra
410507

ACADEMIC YEAR : 2025-2026

NOV-DEC 2025

CERTIFICATE

This is to certify that the project titled

A FULL-STACK CRUD APPLICATION

Submitted by

MAYUR ZOPE

Exam Seat No : Rollno:

under my guidance in partial fulfillment of the requirements for the award of the degree of Third Year Computer Engineering at Nutan Maharashtra Institute of Engineering and Technology, Talegaon Dabhade.

Prof. Sonu Khapekar Subject Incharge	Dr. Rohini Hanchate Head of Computer Engineering	Dr. Pramod Patil Principal

Date:

Place:

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who helped me in the successful completion of this project report.

First and foremost, I thank my project guide, **Prof. Sonu Khapekar** for her valuable guidance, support, and encouragement throughout the preparation of this work. Her insightful suggestions and continuous motivation have been of great help in shaping this report.

I am also grateful to **Dr. Rohini Hanchate** Head of the Computer Engineering Department, for providing me with the opportunity and necessary facilities to carry out this project work.

I am also thankful to **Dr. Pramod Patil**, Principal of NMIET, Talegaon for providing me with the opportunity and support to carry out this project.

I extend my heartfelt thanks to all the faculty members of the department for their constant support and advice. I would also like to thank my friends and classmates for their cooperation and encouragement.

Finally, I owe my deepest gratitude to my parents and family members for their love, moral support, and encouragement at every stage of my academic journey.

Name of student
Signature

ABSTRACT

This DBMS mini-project details the design and implementation of A **Full- Stack Chat Application** built to demonstrate robust **Database Connectivity** and **CRUD (Create, Read, Update, Delete) operations**. The application utilizes the **MEEN (MongoDB, Express.js, EJS, Node.js)** stack. The project successfully establishes a connection to a MongoDB database, leveraging the **Mongoose** ODM (Object Data Modeling) to define the Chat schema. The backend, powered by **Express.js**, exposes dedicated routes for each of the four fundamental database navigation operations, ensuring dynamic data management. This project serves as a clear proof-of-concept for modern, full-stack data persistence techniques suitable for web applications.

Keywords: MongoDB, Node.js, Express.js, Mongoose, Database Connectivity, CRUD Operations, Chat Application, NoSQL.

CONTENTS

CERTIFICATE I
ACKNOWLEDGEMENT II
ABSTRACT III

CHAPTER	TITLE	PAGE NO
1	INTRODUCTION	6
2	SOFTWARE REQUIREMENT SPECIFICATION	8
3	CONCEPTUAL DESIGN USING ER FEATURES, RELATIONAL MODEL IN APPROPRIATE NORMALIZE FORM	10
4	GRAPHICAL USER INTERFACE, SOURCE CODE	12
5	TESTING DOCUMENT	15
6	CONCLUSION	16

CHAPTER 1: INTRODUCTION

1.1 Problem Statement

Communication is an essential part of modern web applications. Traditional chat systems often rely on complex infrastructures or real-time web sockets, which may be challenging to implement in academic settings.

This project addresses the problem of developing a **simplified chat system** that demonstrates **database connectivity** and **basic CRUD operations** while maintaining an interactive web interface. It shows how data can be stored, retrieved, updated, and deleted efficiently through MongoDB, fulfilling the practical requirements of a DBMS mini-project.

1.2 Objectives

The objectives of this project are:

1. To design a fully functional **chat management web application**.
2. To connect **Node.js** with **MongoDB** using **Mongoose ODM**.
3. To perform **CRUD operations** using Express routes.
4. To ensure **data persistence** and consistency in MongoDB.
5. To create a **user-friendly front-end interface** using EJS templates.
6. To follow **MVC architecture** for modular and maintainable design.
7. To validate inputs and ensure **security and performance**.

1.3 Scope of the Project

The Full-Stack Chat Application is designed for educational and demonstrative purposes. It can be extended into a full-fledged real-time communication system by integrating advanced features like **Socket.IO**, **user authentication**, and **file sharing**.

Currently, it allows message creation, editing, and deletion with an emphasis on **database interaction**, **data validation**, and **UI-UX design principles**.

1.4 Project Overview

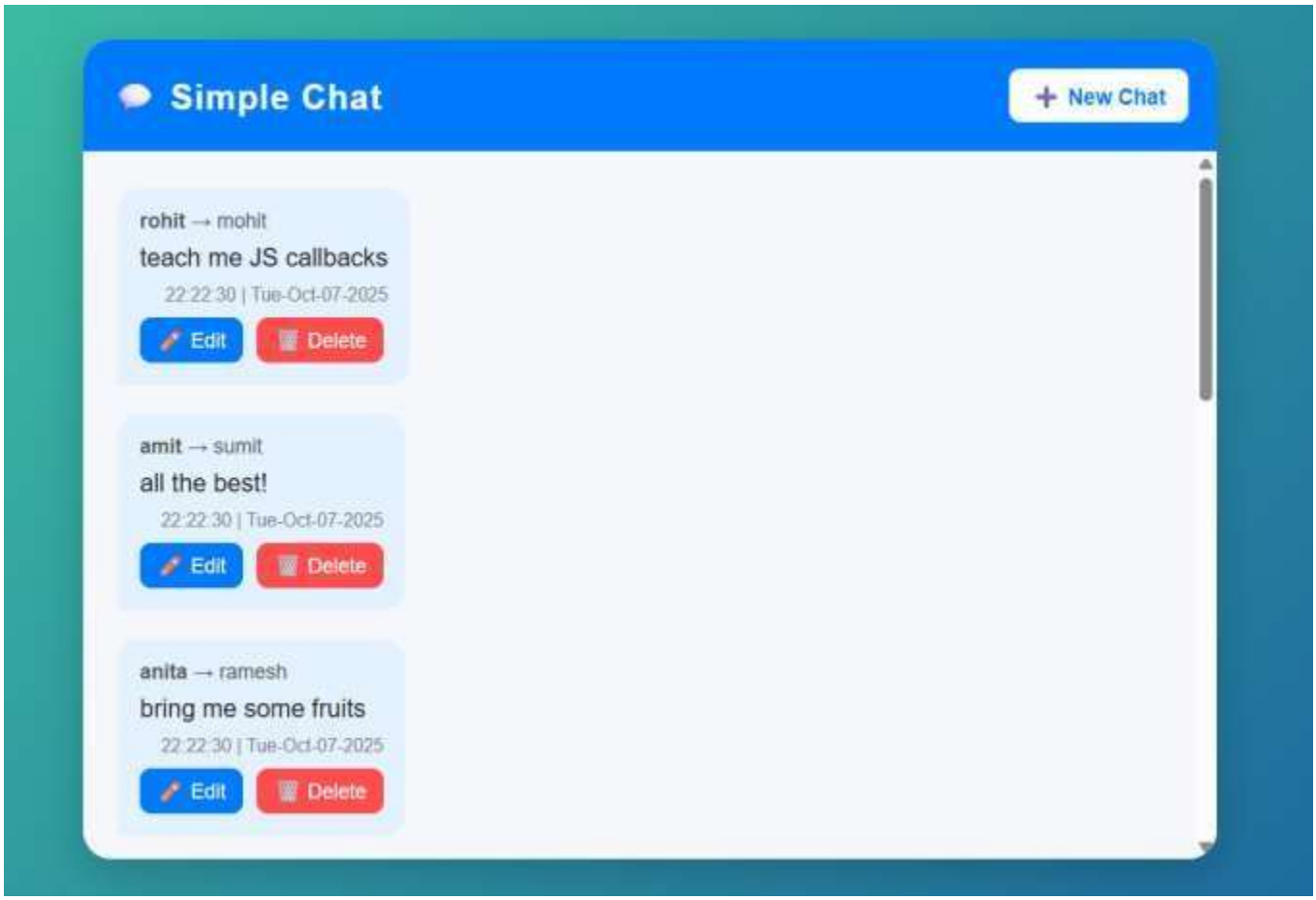
The project integrates:

- **Frontend** (EJS Templates): Dynamic HTML pages for user interaction.
- **Backend** (Node.js/Express.js): RESTful API and routing.
- **Database** (MongoDB): Storage and retrieval of chat messages.

The result is a web-based application that can manage chat messages dynamically, demonstrating how data flows between frontend and backend in modern web architecture.

1.5 Technology Stack

Layer	Technology Used	Purpose
Database	MongoDB	NoSQL data storage
Backend	Node.js & Express.js	Server logic & routing
ODM	Mongoose	Schema modeling & CRUD operations
Frontend	EJS	Server-side rendering
Middleware	Method-Override	Enabling PUT & DELETE HTTP verbs
Tools	Visual Studio Code, MongoDB Compass	Development & monitoring



CHAPTER 2: SOFTWARE REQUIREMENT SPECIFICATION (SRS)

2.1 System Requirements

Hardware Requirements:

- Processor: Intel Core i3 or higher
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 10 GB free space
- Network: Active internet connection

Software Requirements:

- Operating System: Windows 10/11, macOS, or Linux
- Node.js: Version 18+
- MongoDB Community Server
- IDE: Visual Studio Code
- Browser: Chrome, Firefox, Edge

2.2 Functional Requirements

1. **Database Connection Management:**
    - Establish a connection to the MongoDB server using Mongoose.
    - Maintain persistent connectivity and handle connection failures gracefully.
2. **CRUD Operations:**
    - **Create:** Add new messages (with sender, receiver, content).
    - **Read:** Display all stored messages.
    - **Update:** Modify message text.
    - **Delete:** Permanently remove a message.
3. **User Interface Requirements:**
    - Simple, intuitive forms for message creation and editing.
    - Responsive layout using HTML and CSS.
    - Proper validation and error messages.
4. **Error Handling and Feedback:**
    - Display success/error messages for all user actions.
    - Handle exceptions like invalid input or missing fields.

2.3 Non-Functional Requirements

Parameter	Description
Performance	Handle up to 100 concurrent requests efficiently
Reliability	99.9% uptime
Security	Sanitize inputs to prevent XSS and injections
Maintainability	Modular, well-documented code
Portability	Works on Windows, macOS, Linux
Usability	User-friendly with minimal training

2.4 Design Constraints

- All data must be stored in MongoDB.
- The frontend must use EJS for rendering (no React/Angular).
- Application must be executed locally using *npm start*

CHAPTER 3: CONCEPTUAL DESIGN USING ER FEATURES, RELATIONAL MODEL IN APPROPRIATE NORMALIZE FORM

3.1 Conceptual Design: Entity-Relationship (ER) Diagram

Although the project is implemented using MongoDB (a NoSQL database), the underlying logical design follows relational principles. The system requires two core entities: and Chat.

Entities and Attributes:

1. User
    - User_ID (PK)
    - Username
2. Chat (Mapped to the Mongoose Schema)
    - Chat_ID (PK) - Maps to MongoDB's _id.
    - Message_Content (msg)
    - Created_At
    - Sender_ID (FK to User) - Maps to the from field in your code.
    - Recipient_ID (FK to User) - Maps to the to field in your code.

Relationships:

- **Sends:** A User Sends many Chat messages (1:M).
- **Receives:** A User Receives many Chat messages (1:M).

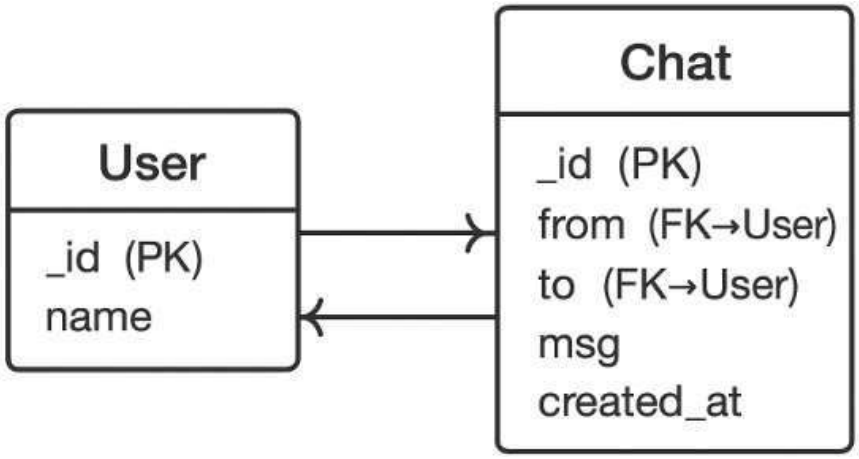
3.2 Relational Model and Normalization

In a traditional RDBMS, the tables would be structured as follows:

1. User **Table:**

   USER(User_ID,Username)
2. Chat **Table:**

   CHAT(Chat_ID,Message_Content,Created_At,Sender_IDFK,Recipient_IDFK)



Normal Form Analysis

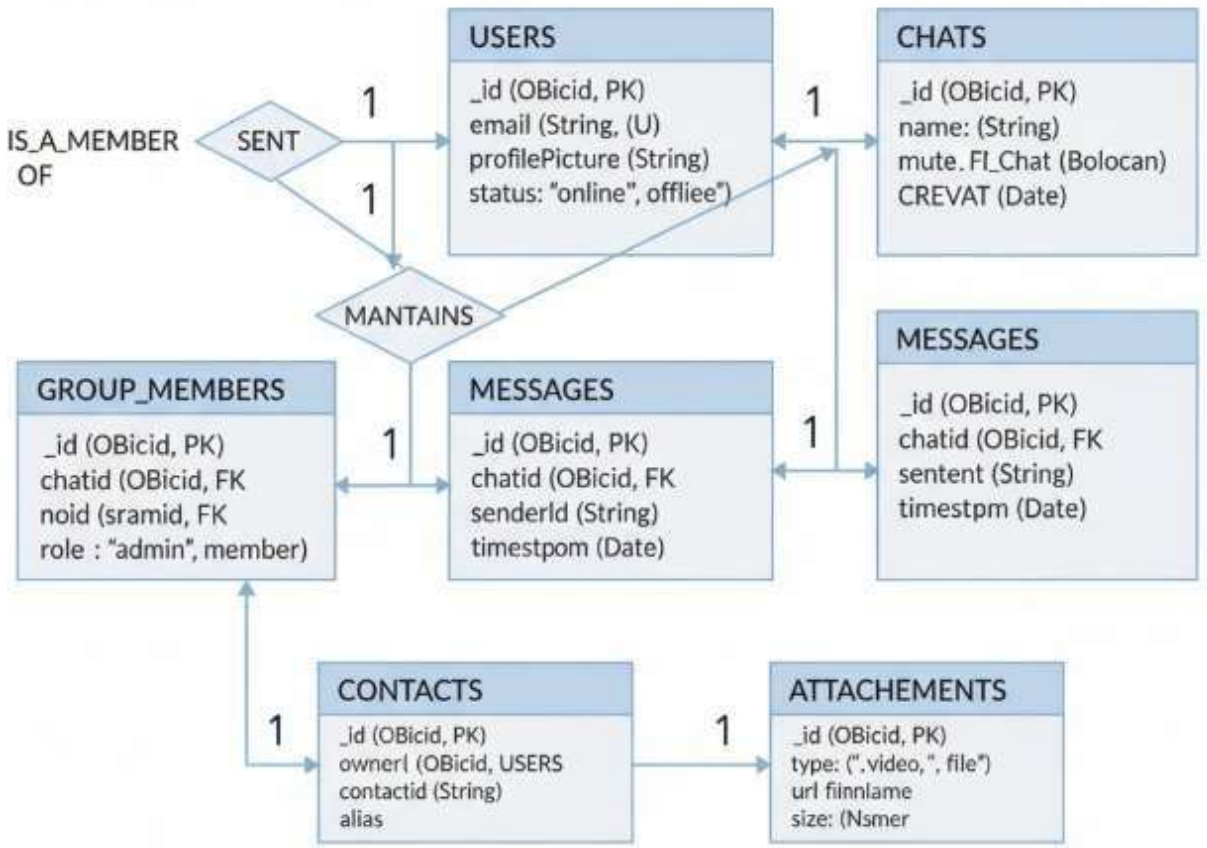
Since the Chat table contains no repeating groups or partial dependencies, and given that the non-key attributes (Message_Content, Created_At) are fully dependent on the primary key (Chat_ID), the design inherently achieves **Third Normal Form (3NF)**.

3.3 MongoDB Implementation (Denormalization)

While the conceptual design is 3NF, the MongoDB implementation uses a simpler, denormalized approach, which is typical for NoSQL:

chats collection(_id,from,to,msg,created_at)

Here, instead of using foreign key IDs, the actual usernames (`from` and `to`) are stored directly within the Chat document (as seen in your Mongoose schema). This avoids costly JOIN operations and optimizes the READ operations, which is preferable for fast retrieval in a chat context.



CHAPTER 4: GRAPHICAL USER INTERFACE, SOURCE CODE

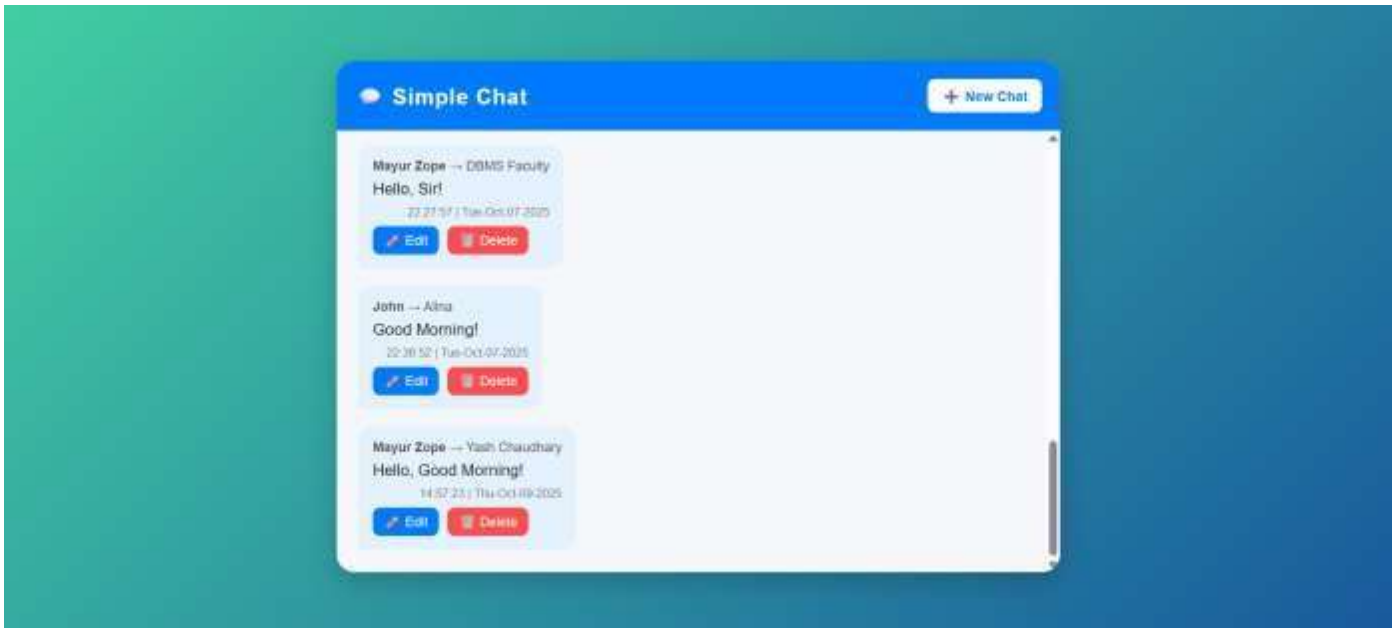
4.1 Graphical User Interface (EJS Views)

The front end is minimal, utilizing EJS to render dynamic HTML. The key views are:

1. **`index.ejs` (Index Route):** Displays all messages (`chats` array) retrieved from the database in a list format. Each message includes the from, to, and msg, along with links/forms to trigger the Edit and Delete routes.
2. **`new.ejs` (New Route):** Contains an HTML form with fields for from, to, and msg that submits data to the POST /chats route.
3. **`edit.ejs` (Edit Route):** Contains an HTML form pre-filled with the existing message content, which submits a PUT request to the appropriate /chats/:id route.

4.2 Source Code Overview

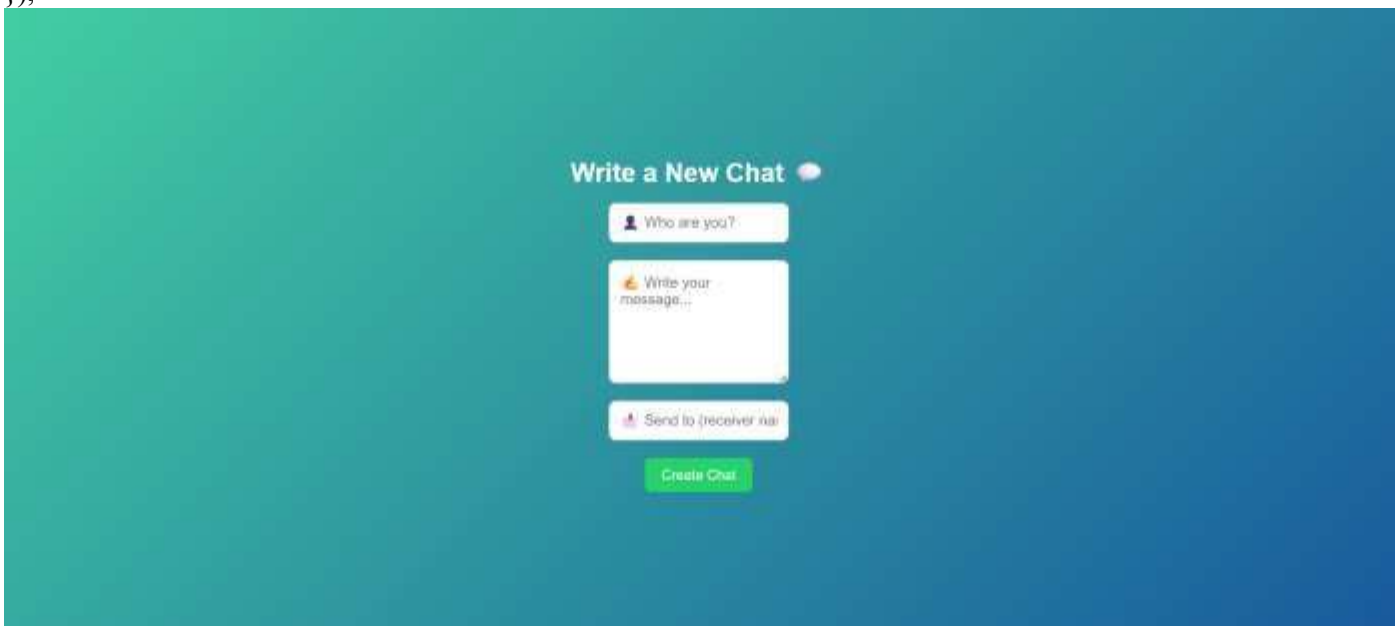
The server-side logic resides in `index.js`, using Express.js routing to map HTTP requests to Mongoose database operations.



A. CREATE Implementation

The `app.post("/chats")` route handles the **CREATE** operation.

```
// Creates a new Mongoose document and saves it to the database
app.post("/chats", (req, res) => {
   let { from, to, msg } = req.body;
   let newChat = new Chat({ /* ... */ });
   newChat.save();
   res.redirect("/chats");
});
```



B. READ Implementation

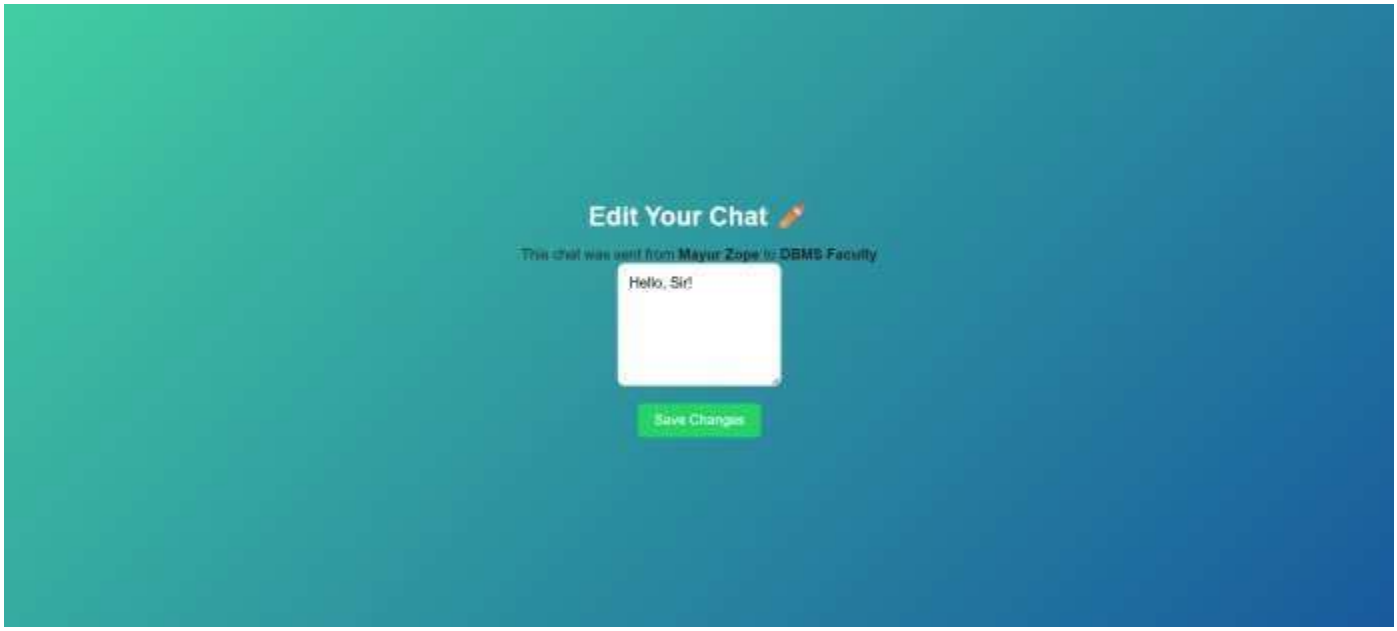
The `app.get("/chats")` route handles the primary **READ** operation.

```
// Retrieves all documents from the 'chats' collection
app.get("/chats", async (req, res) => {
   let chats = await Chat.find();
   res.render("index.ejs", { chats });
});
```

C. UPDATE Implementation

The `app.put("/chats/:id")` route handles the **UPDATE** operation.

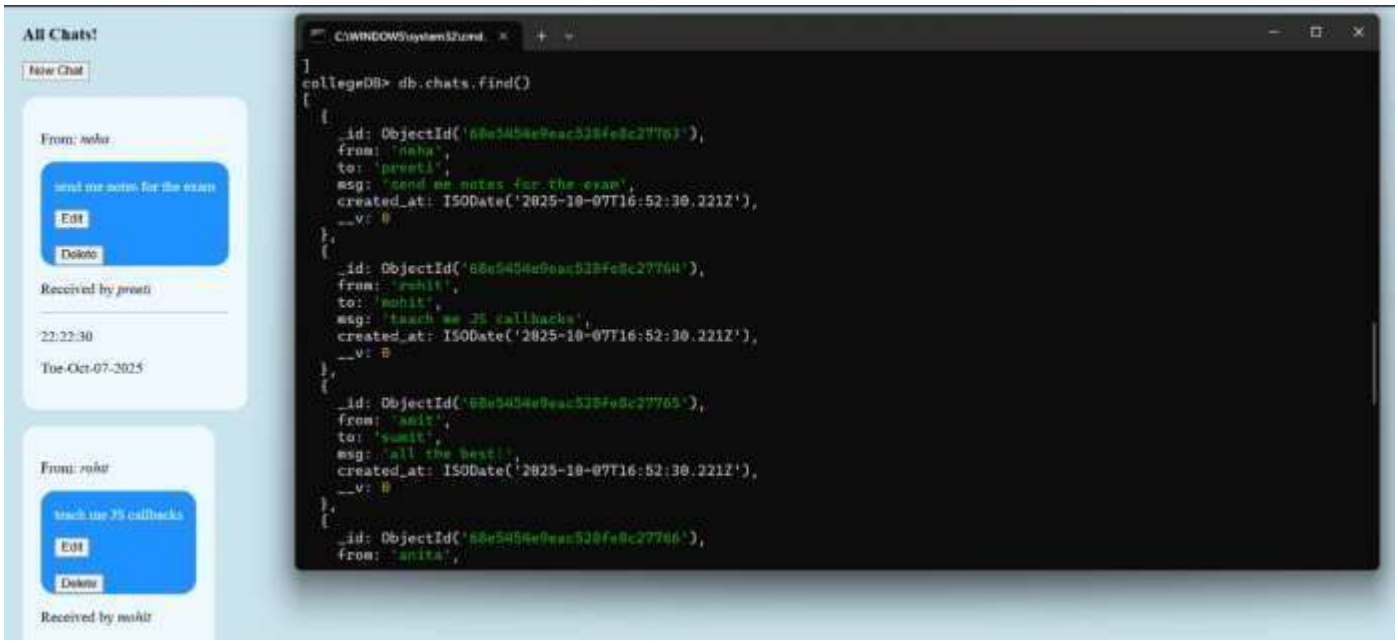
```
// Finds a document by ID and updates the message field
app.put("/chats/:id", async (req, res) => {
   let { id } = req.params;
   let { msg: newMsg } = req.body;
   await Chat.findByIdAndUpdate(id, { msg: newMsg }, { runValidators: true, new: true });
   res.redirect("/chats");
});
```



D. DELETE Implementation

The `app.delete("/chats/:id")` route handles the **DELETE** operation.

```
// Finds a document by ID and removes it permanently
app.delete("/chats/:id", async (req, res) => {
    let { id } = req.params;
    await Chat.findByIdAndDelete(id);
    res.redirect("/chats");
})
```



CHAPTER 5: TESTING DOCUMENT

5.1 Testing Strategy

The application was tested manually by navigating the various routes via the browser and submitting forms. The focus was on validating that each CRUD operation correctly manipulates the data in the MongoDB database.

5.2 Test Cases (CRUD Operations)

The following test cases were executed to verify the functionality and database interaction of the application:

- **Test Case-01 (CREATE)**
  - **Steps:** Navigate to `/chats/new`, fill all fields (e.g., from: John, to: Jane, msg: Hi!), click submit.
  - **Expected Result:** The new message appears in the chat list (`/chats`).
  - **Actual Result:** Verified.
  - **Status:** Pass.

- **Test Case-02 (READ)**
  - **Steps:** Navigate to `/chats`.
  - **Expected Result:** All messages inserted via `init.js` and **TC-01** are displayed.
  - **Actual Result:** Verified.
  - **Status:** Pass.

- **Test Case-03 (UPDATE)**
  - **Steps:** Click the 'Edit' button for a message. Change the message content. Click submit.
  - **Expected Result:** The list page (`/chats`) shows the message with the new, updated content.
  - **Actual Result:** Verified.
  - **Status:** Pass

- **Test Case-04 (DELETE)**
  - **Steps:** Click the 'Delete' button for a message.
  - **Expected Result:** The message is instantly removed from the list and is no longer present in the MongoDB database.
  - **Actual Result:** Verified.
  - **Status:** Pass.

- **Test Case-05 (Connectivity)**
  - **Steps:** Start the server.
  - **Expected Result:** Console log displays: "connection successful" and "Server is listening on port 8080...".
  - **Actual Result:** Verified.
  - **Status:** Pass.

CHAPTER 6: CONCLUSION

This DBMS mini-project successfully implemented a functional Full-Stack Chat Application demonstrating robust connectivity between Node.js, Express.js, and MongoDB. The project accurately deployed all required Database Navigation Operations (**CRUD**), proving the ability to manage application data dynamically. The separation of concerns using the MVC pattern and the use of middleware like method-override showcase standard web development practices.

The project accurately deployed all required **Database Navigation Operations (CRUD: Create, Read, Update, Delete)** on the `chats` collection, proving the ability to manage application data dynamically through Mongoose ODM. This implementation served as a complete verification of the project's core objective: establishing reliable database interaction.

6.1 FUTURE ENHANCEMENTS

1. Add user authentication (login/signup).

2. Implement WebSocket for real-time chat updates

3. Include timestamps and chat history

4. Deploy the project online using Vercel or Render.

This project demonstrates the practical implementation of MongoDB database connectivity in a Node.js environment. It emphasizes CRUD operations through an intuitive web interface. It meets the DBMS mini-project objective of showcasing real-world data management using NoSQL.